Họ và tên: Phan Khánh Vương | **MSSV:** 2312802
Học phần: Kiểm thử Phần mềm | **Ngày thực hành:** 16/10/2026

Chữ ký xác thực
Phan Khánh Vương

TRANG 1: SƠ ĐỒ LUỒNG KIẾN TRÚC CI PIPELINE

Sơ đồ khối mô tả kiến trúc đường ống CI Pipeline tự động hóa với GitHub Actions (gồm đầy đủ 6 khối chức năng cốt lõi):

1. SỰ KIỆN KÍCH HOẠT (TRIGGER EVENT)
Sự kiện push hoặc pull_request vào nhánh chính (main)

↓

2. CHECKOUT MÃ NGUỒN (CHECKOUT CODE)
actions/checkout@v4 → Tải toàn bộ mã nguồn về máy chủ ảo sạch (Ubuntu runner)

↓

3. THIẾT LẬP MÔI TRƯỜNG PYTHON (SETUP ENVIRONMENT)
actions/setup-python@v5 → Cài đặt chính xác Python phiên bản 3.11

↓

4. CÀI ĐẶT THƯ VIỆN PHỤ THUỘC (INSTALL DEPENDENCIES)
pip install -r requirements.txt → Cài đặt thư viện đã ghim phiên bản (requests, coverage, flake8)

↓

5. KIỂM TRA CÚ PHÁP TĨNH (STATIC CODE ANALYSIS - LINTER)
flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
Nguyên lý Fail-Fast: Nếu phát hiện lỗi cú pháp, Exit Code ≠ 0 → **BREAK THE BUILD NGAY LẬP TỨC**

↓ **(Linter sạch: Exit Code = 0)**

6. THỰC THI KIỂM THỬ VÀ CỔNG CHẤT LƯỢNG (TESTS & QUALITY GATE)
Execute Unit Tests (Mocking GPS) → coverage report --fail-under=80
Quality Gate: Nếu Coverage < 80% → Exit Code = 2 → **BUILD FAILED (BÁO ĐỎ)**
Nếu Coverage ≥ 80% → Exit Code = 0 → **BUILD PASSED (TÍCH XANH)**

↓

7. LƯU TRỮ VÀ XUẤT ARTIFACTS (UPLOAD HTML REPORT)
actions/upload-artifact@v4 (if: always()) → Lưu trữ báo cáo coverage_html/ trong 7 ngày

Họ và tên: Phan Khánh Vương | **MSSV:** 2312802
Học phần: Kiểm thử Phần mềm | **Ngày thực hành:** 16/10/2026

Chữ ký xác thực
Phan Khánh Vương

TRANG 2: BẢNG MA TRẬN THIẾT KẾ KỊCH BẢN TEST CASE MOCKING

Ma trận thiết kế chi tiết 7 kịch bản kiểm thử ngoại vi cho dịch vụ mạng và phần cứng GPS:

Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)
1. test_apply_promo_online_valid	`apply_promo_code`	Không mock. Trạng thái: `network_status="online"`, `os_version=15`, mã `"GIAM10"`	Áp dụng giảm 10% thành công. `assertEqual(result, 90000.0)`
2. test_apply_promo_offline_raises_connection_error	`apply_promo_code`	Không mock. Trạng thái: `network_status="offline"`, `os_version=15`, mã `"GIAM10"`	Bị chặn do mất kết nối mạng. `assertRaises(ConnectionError)`
3. test_apply_promo_special_code_os15	`apply_promo_code`	Không mock. Trạng thái: `network_status="online"`, `os_version=15`, mã `"NEW_OS_ONLY"`	Áp dụng giảm 20% thành công trên OS 15. `assertEqual(result, 80000.0)`
4. test_apply_promo_special_code_os14_raises_value_error	`apply_promo_code`	Không mock. Trạng thái: `network_status="online"`, `os_version=14`, mã `"NEW_OS_ONLY"`	Ném ngoại lệ do phiên bản OS không đủ (cần ≥ 15). `assertRaises(ValueError)`
5. test_get_nearby_restaurants_in_range	`get_nearby_restaurants`	Mock `gps_service.get_current_location`: `return_value = (10.0, 105.0)`	Trả về danh sách 2 nhà hàng tương ứng tọa độ: `["Nha hang Pho Ly Quoc Su", "Bun cha Huong Lien"]`
6. test_get_nearby_restaurants_out_of_range	`get_nearby_restaurants`	Mock `gps_service.get_current_location`: `return_value = (20.0, 110.0)`	Tọa độ không có đối tác, trả về danh sách rỗng: `assertEqual(restaurants, [])`
7. test_get_nearby_restaurants_signal_lost	`get_nearby_restaurants`	Mock `gps_service.get_current_location`: `side_effect = GPSSignalLostError()`	Bắt ngoại lệ mất GPS và trả về chuỗi thông báo: `"Khong the xac dinh vi tri."`

Họ và tên: Phan Khánh Vương | **MSSV:** 2312802
Học phần: Kiểm thử Phần mềm | **Ngày thực hành:** 16/10/2026

Chữ ký xác thực
Phan Khánh Vương

TRANG 3: LỜI GIẢI CÂU HỎI PHÂN TÍCH CẤP ĐỘ 2

Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

Lời giải:

1. **Thực thi nguyên lý "Thất bại sớm" (Fail-Fast):** Linter (flake8) thực hiện phân tích mã tĩnh (Static Code Analysis) nhằm phát hiện ngay các lỗi cú pháp nghiêm trọng (SyntaxError, sai thụt lề IndentationError, gọi biến chưa khai báo F82, import dư thừa...). Quá trình quét tĩnh này không cần thực thi code nên diễn ra cực kỳ nhanh (chỉ mất 1 - 2 giây), nhẹ hơn rất nhiều so với việc khởi chạy môi trường máy ảo và thực thi toàn bộ bộ Unit Test.
2. **Tiết kiệm tối đa tài nguyên và thời gian máy chủ CI:** Nếu mã nguồn có lỗi cú pháp, bộ test chắc chắn sẽ sập (crash) hoặc thất bại. Việc đặt Linter trước giúp phát hiện và dừng pipeline ngay lập tức, tránh lãng phí thời gian runner và tiết kiệm hạn ngạch tính toán (GitHub Actions minutes).
3. **Bảo đảm chuẩn mực phong cách mã nguồn:** Đảm bảo code tuân thủ nghiêm ngặt chuẩn PEP 8 trước khi tiến hành xác minh nghiệp vụ.

Câu 2.2: Tham số --fail-under=80 của lệnh coverage report hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?

Lời giải:

1. **Nguyên lý Mã thoát (Exit Code / Return Code) trong POSIX/Linux:** Trong hệ điều hành Linux, mỗi tiến trình (process) khi kết thúc đều gửi về cho tiến trình cha một giá trị số nguyên nguyên thủy gọi là Exit Code:
   - Mã `0`: Đại diện cho trạng thái thực thi thành công hoàn toàn (Success).
   - Mã khác `0` (Non-zero, từ 1 đến 255): Đại diện cho lỗi hoặc điều kiện thất bại (Failure).
2. **Cơ chế hoạt động của tham số `--fail-under=80`:** Khi công cụ `coverage` tổng hợp báo cáo:
   - Nếu tổng tỷ lệ bao phủ mã nguồn ≥ 80%: Công cụ kết thúc bình thường và trả về Exit Code = `0`.
   - Nếu tỷ lệ bao phủ < 80%: Công cụ in thông báo lỗi và chủ động gọi `sys.exit(2)` để trả về Exit Code = `2` (khác 0).
3. **Cơ chế dừng tiến trình của GitHub Actions:** Mỗi step shell trên runner Ubuntu mặc định hoạt động với cơ chế kiểm tra biến trạng thái `$?` (tương đương lệnh `set -e`). Khi nhận Exit Code khác 0, runner lập tức đánh dấu step và job đó là FAILED (Báo Đỏ) và hủy toàn bộ các bước tiếp theo.

Câu 2.3: Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử khi tối ưu độ bao phủ mã nguồn?

Lời giải:

- **Thông tin hiển thị:** Tham số `-m` (viết tắt của `--show-missing`) bổ sung thêm cột **"Missing"** trong bảng thống kê ở terminal. Cột này chỉ rõ danh sách số dòng (Line Numbers) hoặc khoảng dòng cụ thể chưa từng được thực thi qua trong lần chạy test (ví dụ: `34-43, 47-52`).
- **Lợi ích đối với Kỹ sư kiểm thử (SET):**
  - **Xóa bỏ phỏng đoán:** Kỹ sư biết chính xác nhánh rẽ nào (nhánh `if`, `else` hay `except`) chưa được kiểm tra mà không phải dò thủ công.
  - **Thiết kế kịch bản có định hướng:** Nhìn vào dòng thiếu để bổ sung đúng test case cần thiết (ví dụ thấy thiếu dòng ném `ConnectionError` thì viết ngay test cho trạng thái offline).
  - **Tối ưu hóa thời gian:** Giúp nâng cao tỷ lệ bao phủ vượt ngưỡng Quality Gate một cách nhanh nhất.

Họ và tên: Phan Khánh Vương | **MSSV:** 2312802
Học phần: Kiểm thử Phần mềm | **Ngày thực hành:** 16/10/2026

Chữ ký xác thực
Phan Khánh Vương

TRANG 4: LỜI GIẢI CÂU HỎI PHÂN TÍCH CẤP ĐỘ 3

Câu 3.1: Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.

Lời giải:

- **Flaky Test (Kiểm thử chập chờn / Không ổn định):**
  - *Định nghĩa:* Là bài test cho ra các kết quả khác nhau (lúc Pass, lúc Fail) dù mã nguồn giữ nguyên và chạy trên cùng một môi trường.
  - *Nguyên nhân:* Do các yếu tố bất định (non-deterministic): tranh chấp tài nguyên đa luồng (race conditions), độ trễ mạng khi gọi dịch vụ ngoài thật (external API timeout, latency), thứ tự thực thi test ngẫu nhiên, hoặc phụ thuộc vào thời gian/múi giờ hệ thống.
- **Lỗi do sai lệch môi trường phụ thuộc (Environment / Dependency Drift):**
  - *Định nghĩa:* Là lỗi xảy ra khi test luôn Pass ổn định trên môi trường này (ví dụ máy local của lập trình viên) nhưng luôn Fail xác định trên môi trường khác (ví dụ máy chủ CI hoặc máy staging).
  - *Nguyên nhân:* Do sự không đồng nhất về cấu hình hệ thống: khác biệt hệ điều hành (Windows vs Linux), sai lệch phiên bản Python (Python 3.10 vs 3.14), hoặc thư viện phụ thuộc có phiên bản khác nhau giữa các máy.
- **Điểm khác biệt cốt lõi:** Flaky Test mang tính bất định (ngẫu nhiên) ngay trên cùng một môi trường (khắc phục bằng Mocking); còn Sai lệch môi trường mang tính xác định giữa các môi trường khác nhau (khắc phục bằng Ghim phiên bản và Container hóa).

Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên bản (Version Pinning) trong requirements.txt?

Lời giải:

1. **Đảm bảo tính tái lập (Reproducibility):** Máy chủ CI là môi trường tạm thời (ephemeral), mỗi lần chạy đều cài mới hoàn toàn thư viện. Ghim phiên bản chuẩn xác (dùng `==`, ví dụ `coverage==7.4.0`) bảo đảm máy chủ CI luôn có môi trường đồng nhất 100% với máy phát triển của lập trình viên.
2. **Ngăn chặn lỗi phá vỡ tương thích (Breaking Changes):** Nếu không ghim phiên bản (hoặc dùng `>=`), khi một thư viện bên thứ ba phát hành bản mới có thay đổi API hoặc lỗi ngầm, đường ống CI sẽ tự động tải bản mới và bị sập ("vỡ build") đột ngột dù mã dự án không hề có lỗi.
3. **Bảo mật chuỗi cung ứng (Supply Chain Security):** Tránh việc CI tự động kéo về các phiên bản độc hại (malicious packages) mới bị phát tán trên PyPI trước khi được đội ngũ phát triển kiểm duyệt.

Câu 3.3: Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

Lời giải:

1. **Bước 1 - Tạo nhánh làm việc độc lập:** Từ nhánh `main` mới nhất, rẽ một nhánh riêng (ví dụ: `git checkout -b chore/upgrade-requests`). Tuyệt đối không nâng cấp trực tiếp trên `main`.
2. **Bước 2 - Cập nhật tệp requirements.txt:** Đổi số phiên bản của thư viện cần nâng cấp trong `requirements.txt` sang bản mới. Đọc kỹ Release Notes để nắm các breaking changes.
3. **Bước 3 - Cài đặt và kiểm thử cục bộ:** Chạy `pip install -r requirements.txt` trên máy cá nhân, thực thi toàn bộ test suite và linter. Nếu có lỗi do đổi API, tiến hành refactor code tương ứng cho đến khi toàn bộ test đều Pass.
4. **Bước 4 - Commit và đẩy nhánh lên GitHub:** Commit thay đổi với thông điệp rõ ràng và đẩy nhánh lên remote repo (`git push origin chore/upgrade-requests`).
5. **Bước 5 - Tạo Pull Request (PR) và xác minh qua CI Pipeline:** Tạo PR vào `main`. Đường ống CI tự động kích hoạt trên máy ảo Ubuntu để kiểm tra linter và test suite. Đảm bảo toàn bộ checks đều hiển thị tích Xanh.
6. **Bước 6 - Review mã nguồn và Merge:** Yêu cầu đồng nghiệp hoặc Tech Lead Code Review. Khi PR được duyệt và CI xanh, tiến hành hợp nhất vào `main` (Squash & Merge) và xóa nhánh tạm.